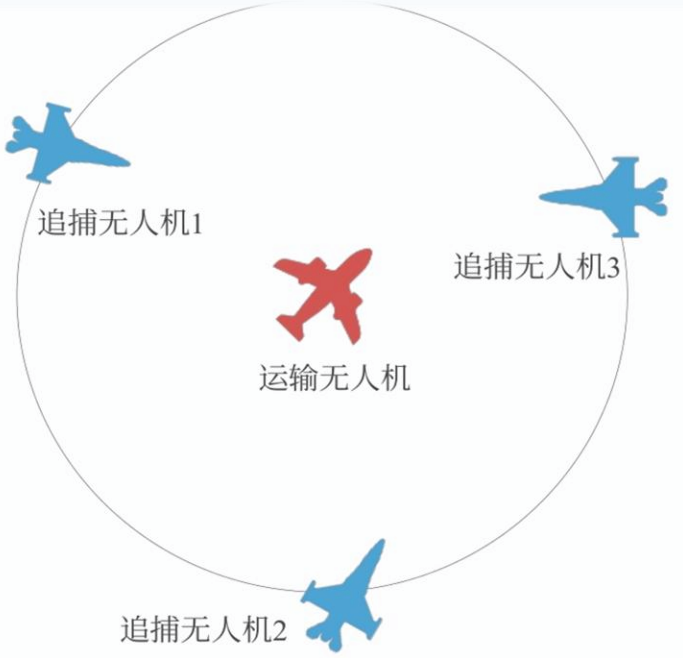
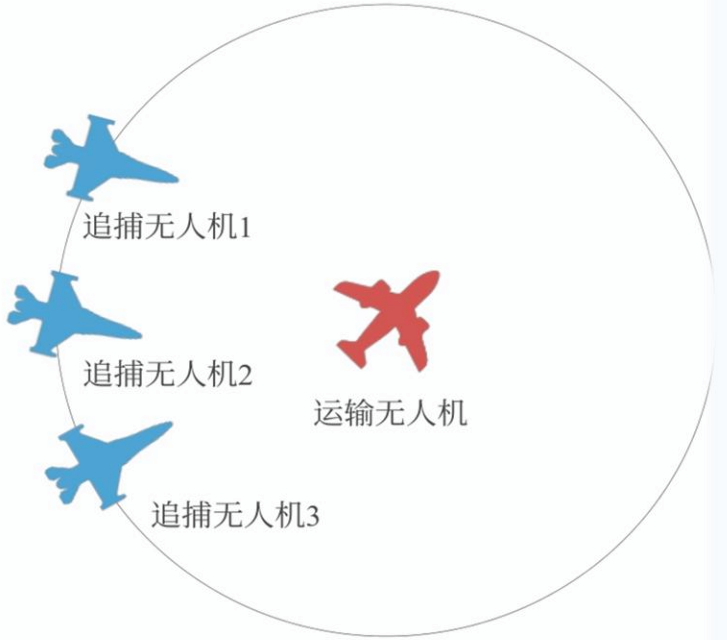
算法驱动无人机群智能围捕动态目标仿真

学号：2553350
姓名：陈跃文
书院：启迪书院
提交时间：2026/3/16

目录



(a) 无人机位于围捕中心



(b) 无人机未在围捕中心

1. 项目背景与系统描述

1.1 研究背景与应用场景

警用安防：江苏睢宁公安低空战队、江西遂川山林围捕案例，无人机集群已成为空地协同抓捕、复杂地形防控的核心装备；

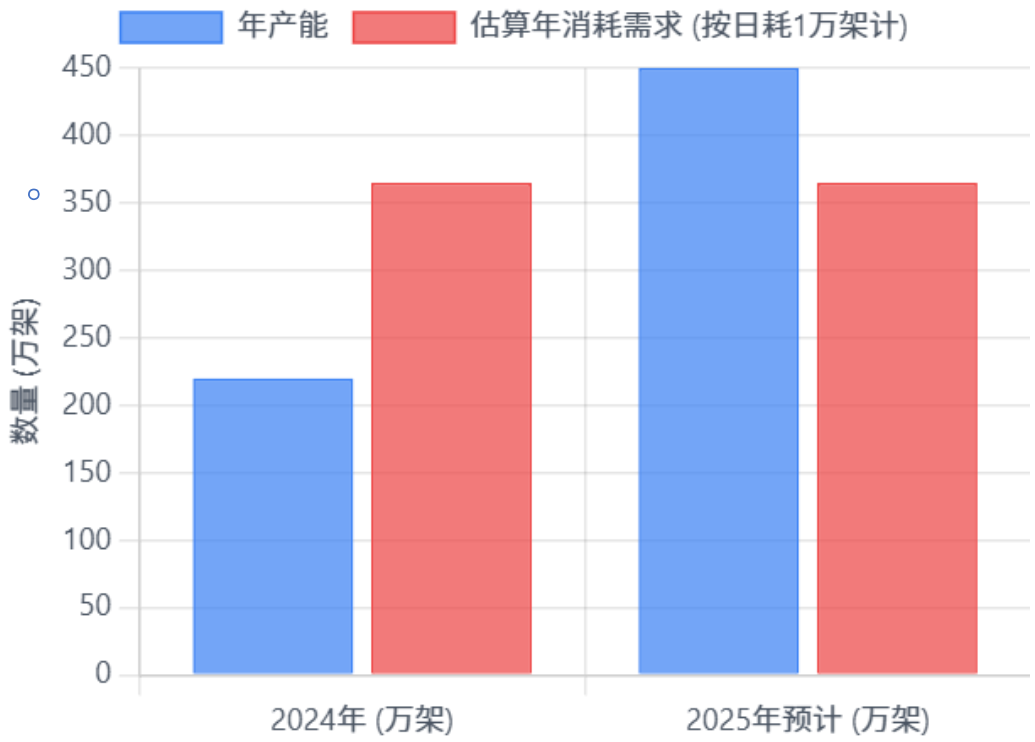
。

军事领域：陆军无人军团蜂群围捕演习，集群协同围堵打击是无人作战的核心能力，对自主化、智能化要求极高。

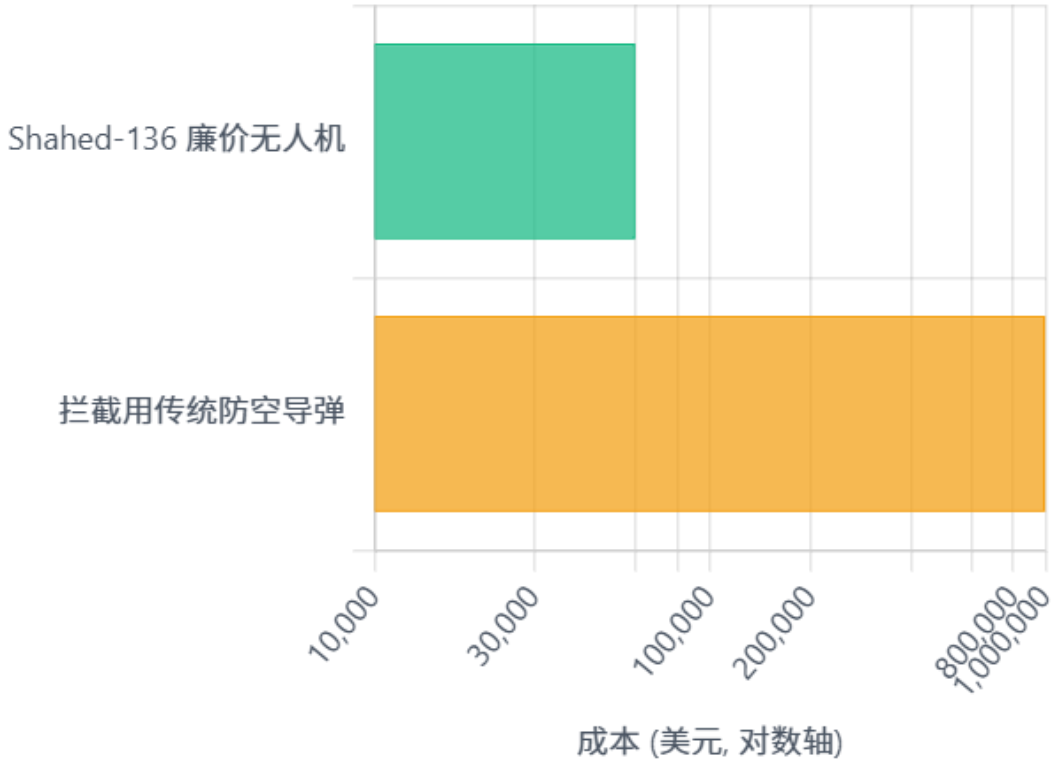
1.1 研究背景与应用场景

无人机应用（以俄乌战争为例）

规模化与高消耗：无人机产能与消耗估算



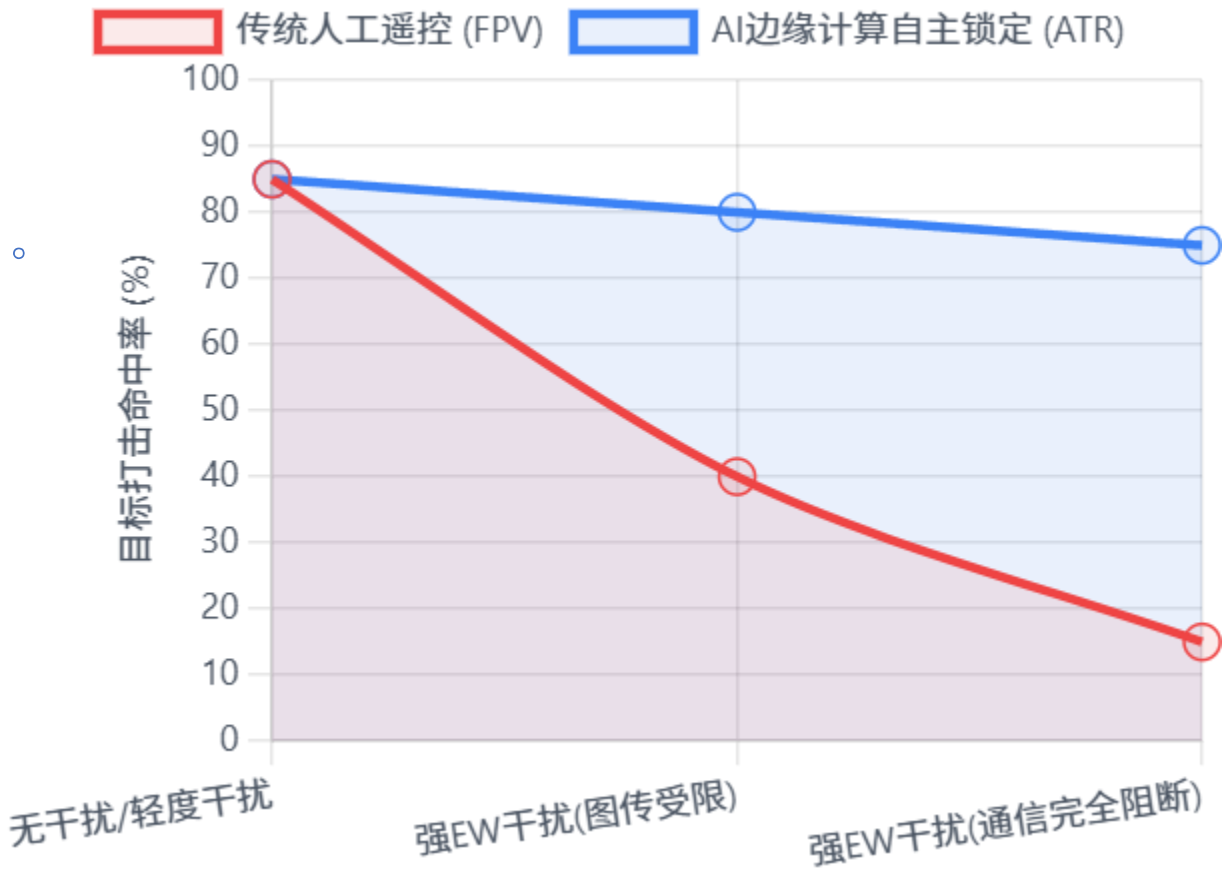
成本非对称：廉价无人机 vs 传统防空



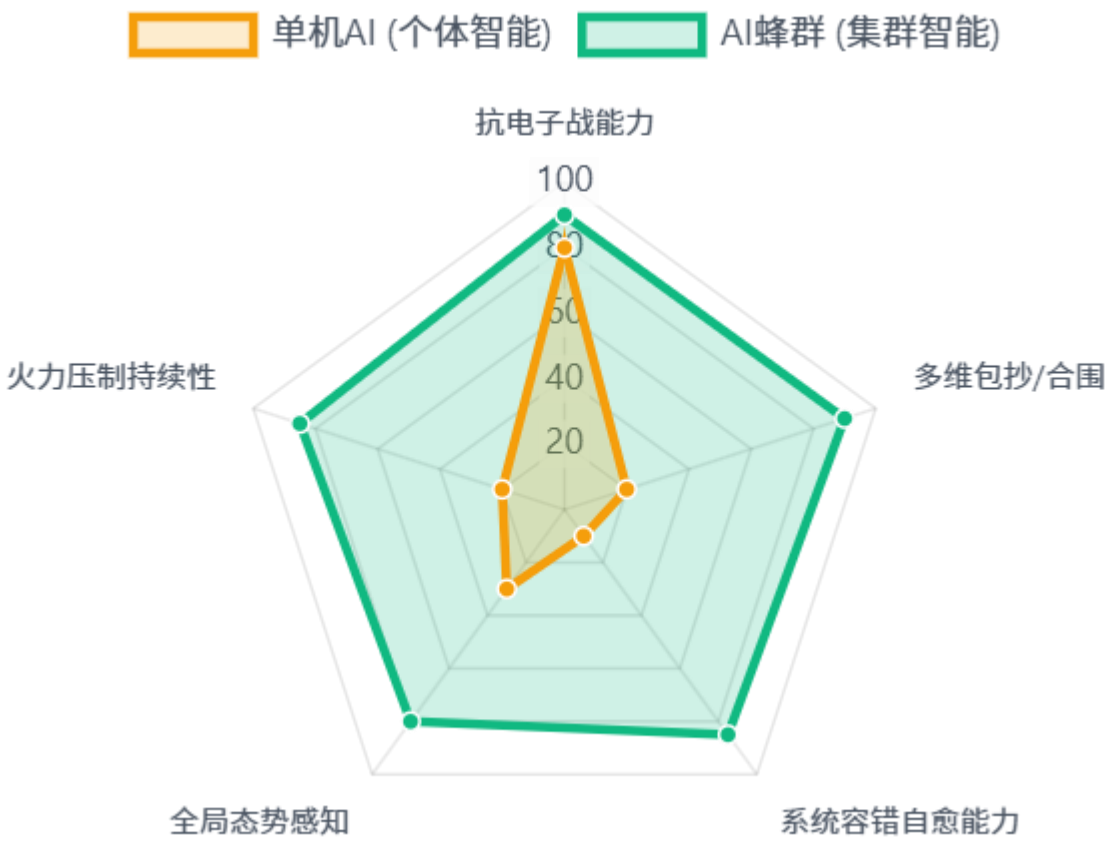
1.1 研究背景与应用场景

无人机应用（以俄乌战争为例）

强电子战(EW)环境对无人机命中率的影响

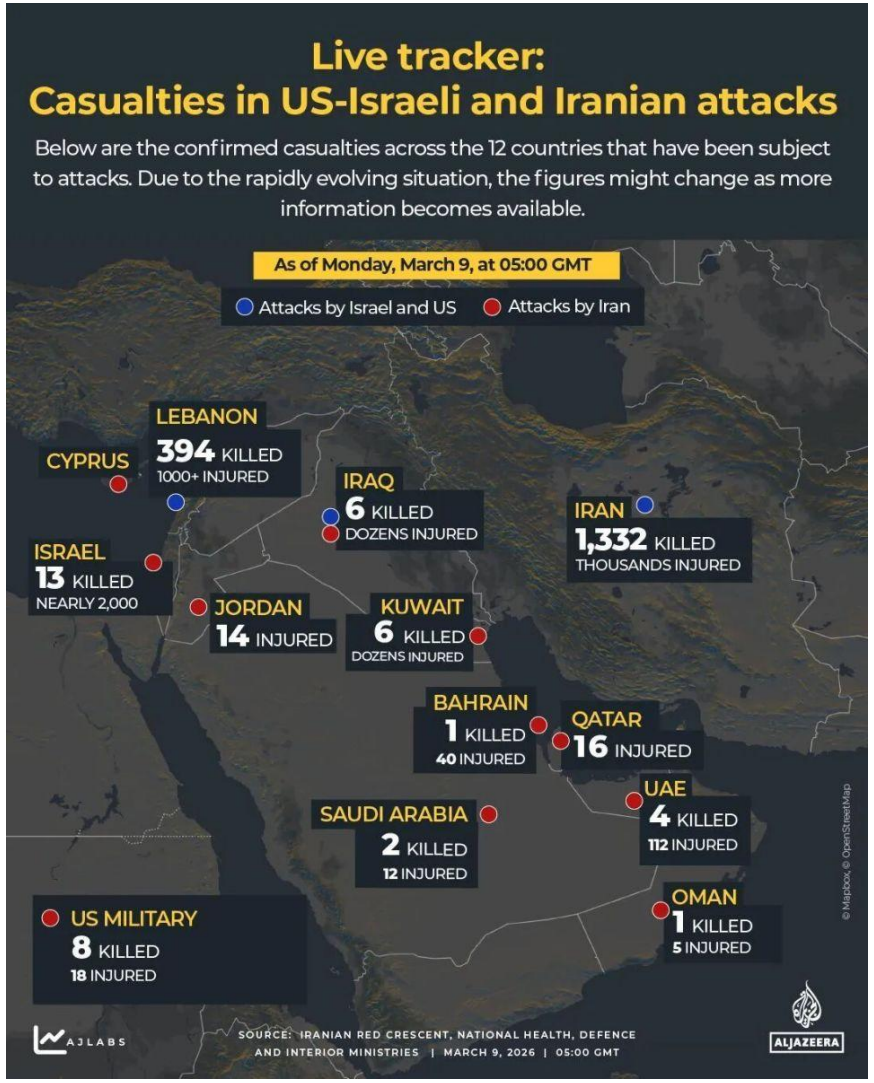


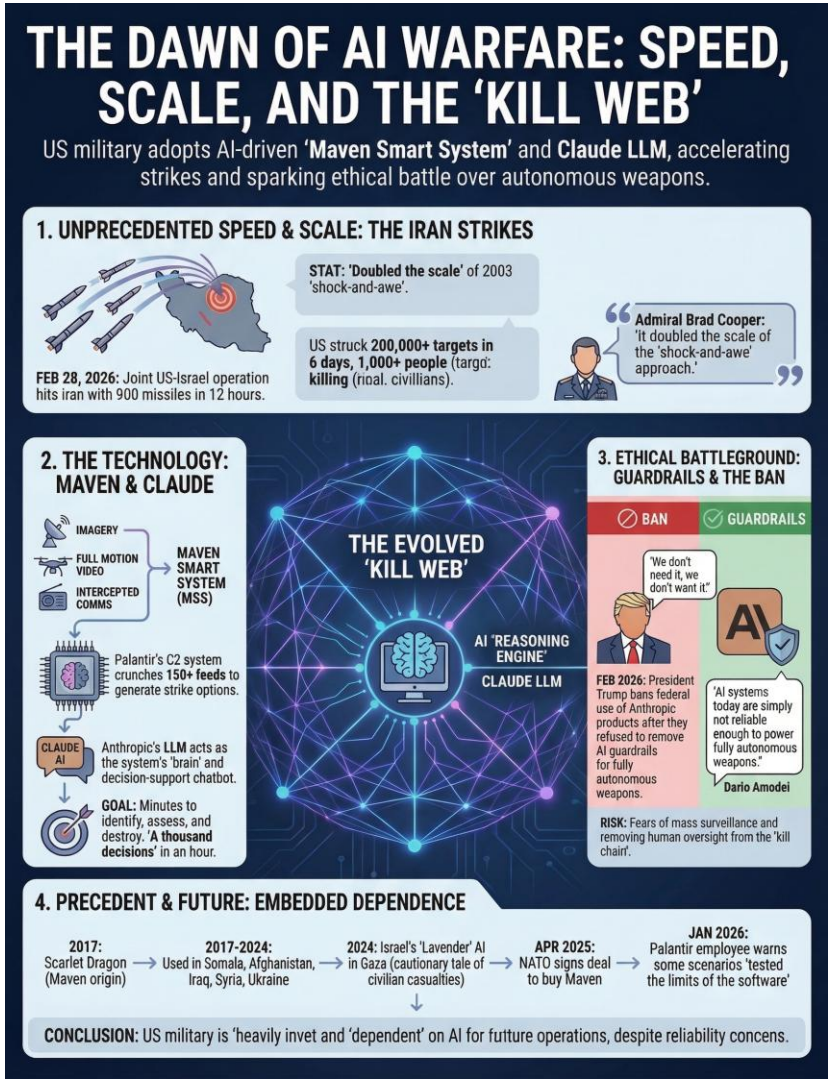
战术演进：单机智能向蜂群协同演进



1.1 研究背景与应用场景

Ai 在军事领域应用（以美以—伊朗战争为例）

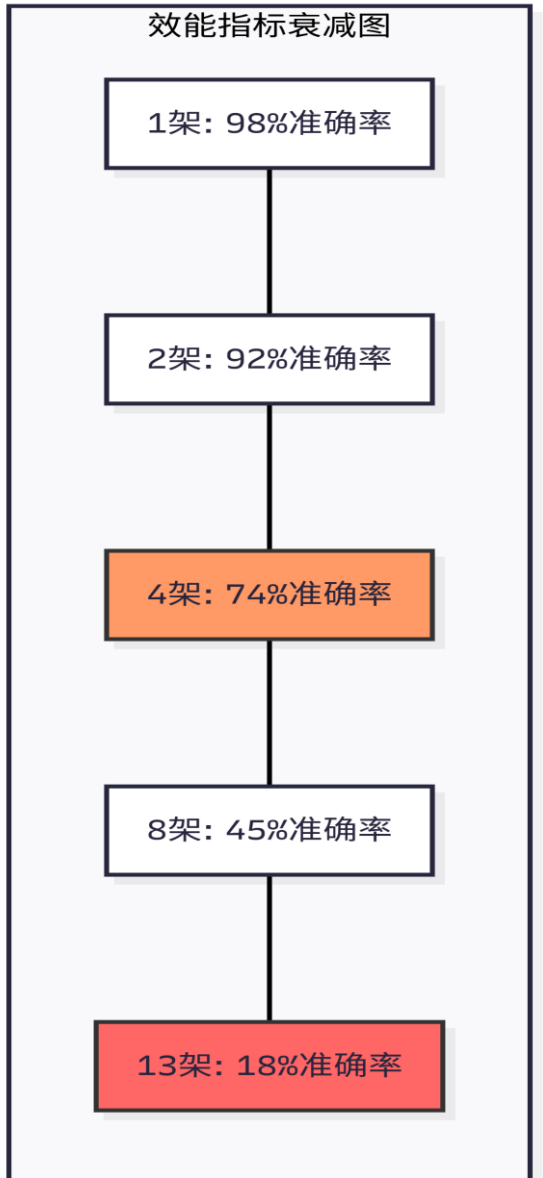




1.2 核心问题与痛点

1. 人工操作模式的“1:N”规模瓶颈

在人工操作模式下，单一操作员控制的无人机数量（N）存在明显的物理极限。随着机群规模扩大，态势感知（SA）得分与任务决策准确率呈现非线性下降。

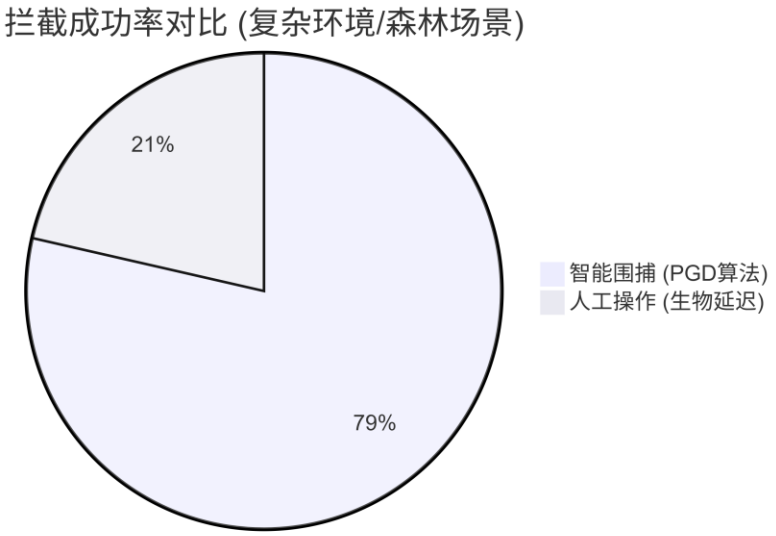


1.2 核心问题与痛点

2. 反应延迟：毫秒级的捕食对弈

在高动态围捕场景中，反应时间决定了拦截的成败。

人工操作受限于生物神经传导延迟，而智能系统利用板载处理可实现亚毫秒级响应。



模式	平均延迟 (ms)
智能围捕 (事件相机+算力优化)	3.5 - 22 ms
传统自主避障 (标准算法)	20 - 40 ms
人工操作 (生物反应链)	200 - 500 ms

1.2 核心问题与痛点

3. 空间安全：机群碰撞风险的二次方增长

在密集围捕圈中，机间碰撞是核心风险点。当缺乏

- 自主协同算法时，碰撞频率随无人机数量呈二次方

增长（f(x) = ax^2 + bx + c）。

1.3 项目目标

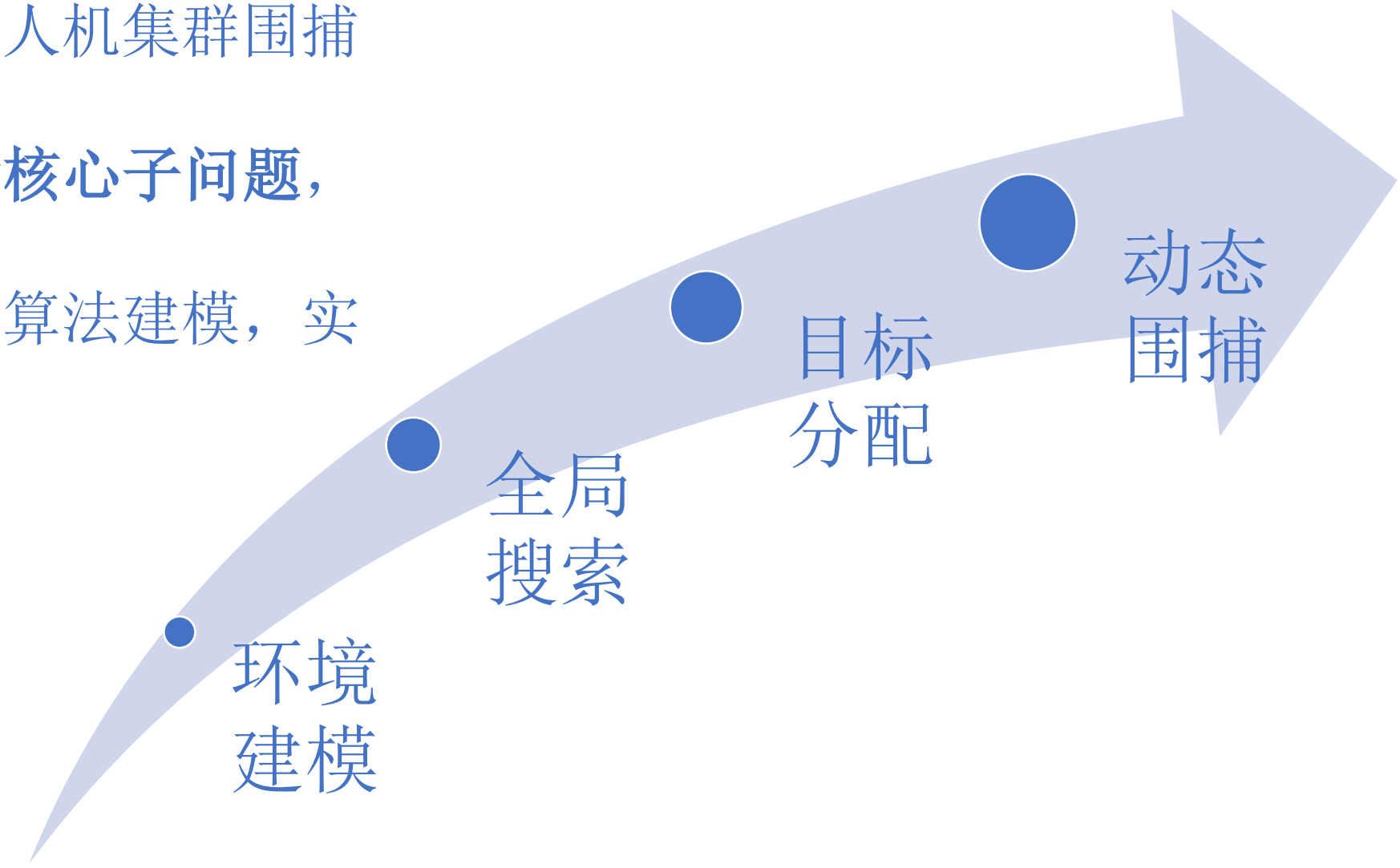
基于元胞自动机、自适应 PSO、匈牙利算法、简易神经网络，搭建无人机集群智能围捕仿真系统，实现未知环境下的自主搜索、智能分配、精准围捕全流程仿真与可视化，验证多智能算法融合的集群围捕性能。

2. 系统核心技术架构

2.1 系统建模思路

本项目将复杂的无人机集群围捕任务，拆解为**四大核心子问题**，

- 针对性完成数学与算法建模，实现全流程闭环



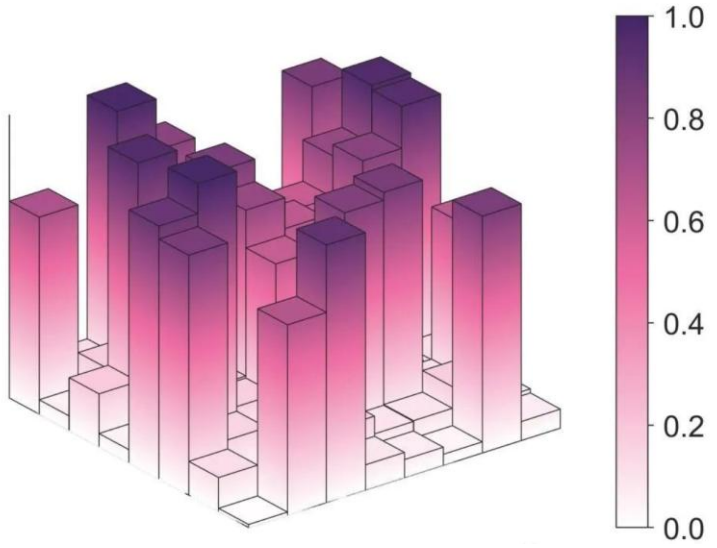
2.1.1 环境搭建（元胞自动机）

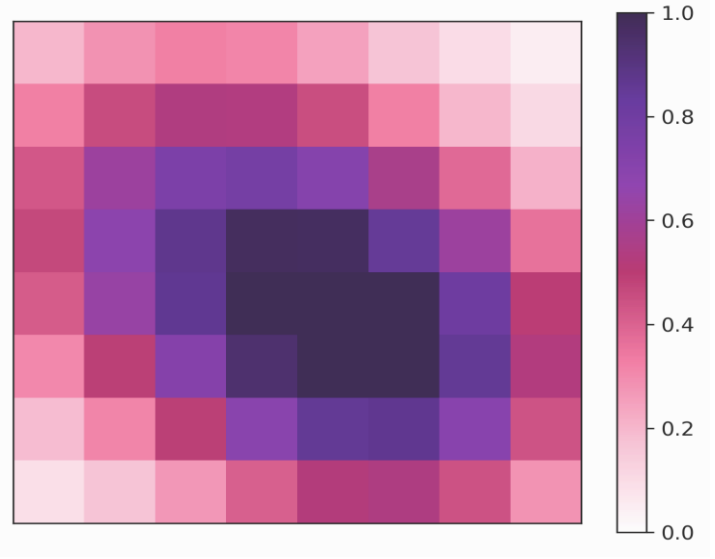
所解决的子问题

未知二维环境的数字化、动态信息传递

建模思路与算法选择

基于元胞自动机，搭建栅格化信息素场，实现障碍物建模、信息素挥发 / 扩散 / 沉积，为无人机提供全局环境引导





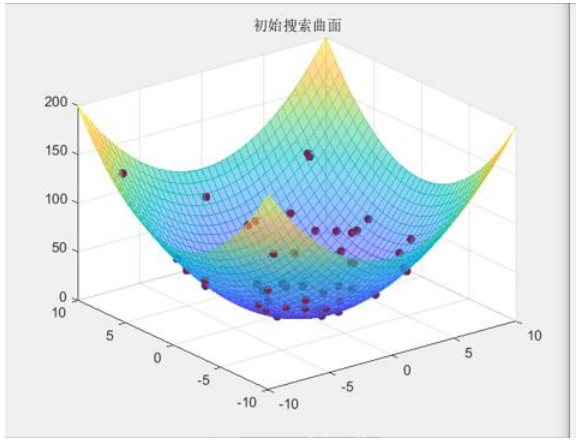
2.1.2 全局搜索（PSO算法）

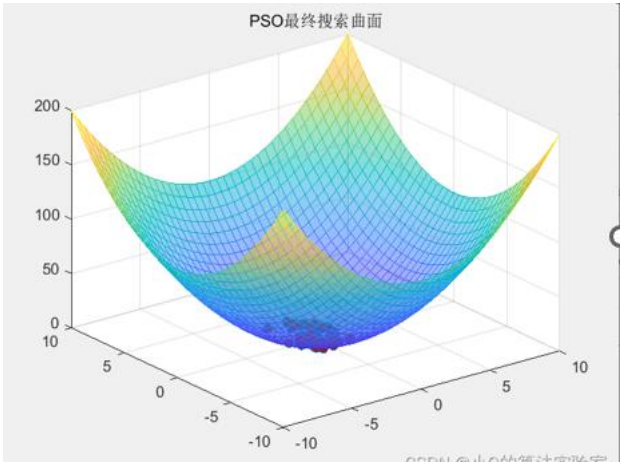
所解决的子问题

未知环境中无人机集群的高效探索、目标线索追踪

建模思路与算法选择

基于自适应权重 PSO 算法，结合信息素场动态调整优化参数，解决传统算法局部最优、搜索效率低的问题从 “盲目搜索” 到 “智能分工”





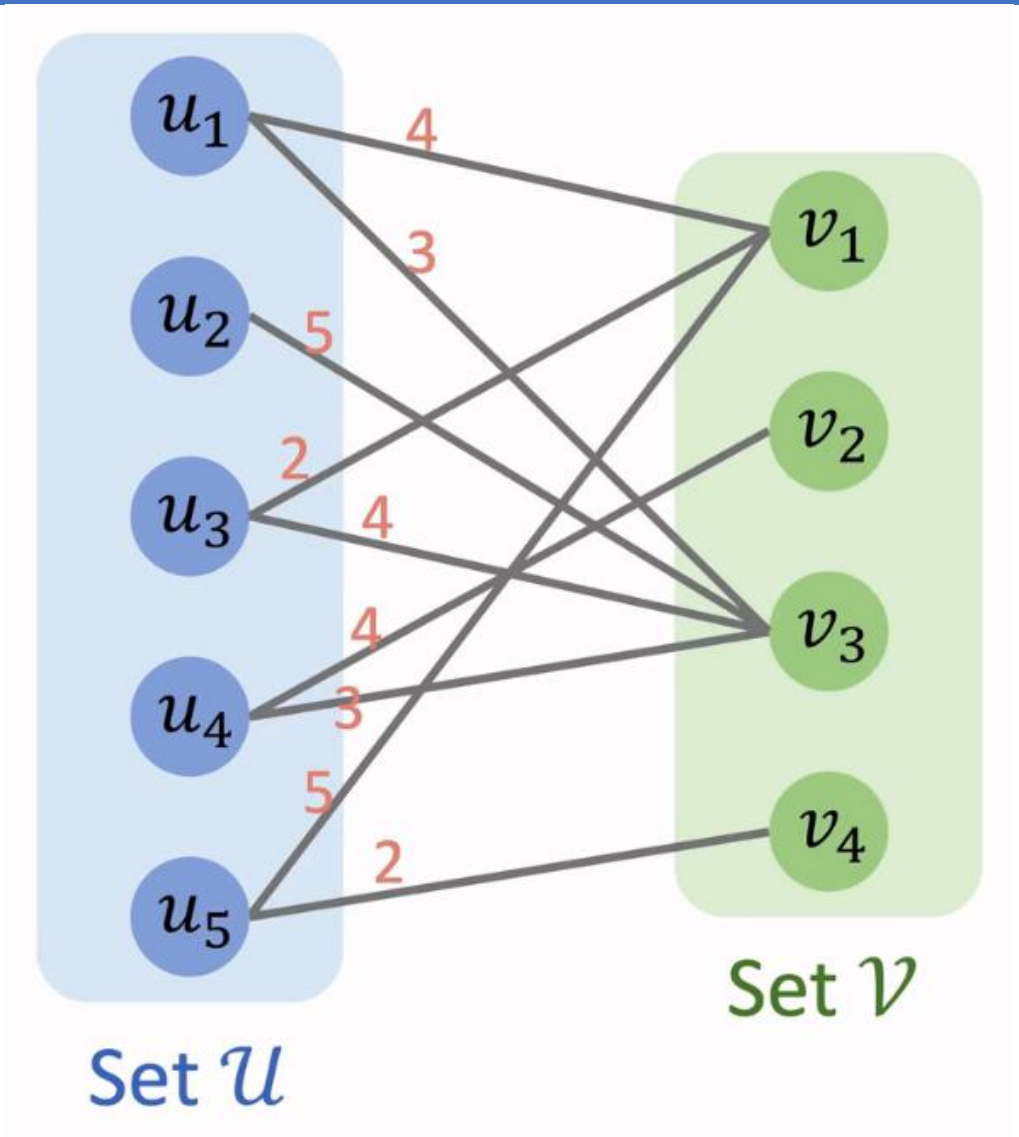
2.1.3 目标分配（匈牙利算法）

所解决的子问题

多无人机 - 多目标的最优匹配，实现智能分工

建模思路与算法选择

基于匈牙利算法，构建 “距离 + 目标优先级” 的代价矩阵，求解全局最优指派方案，避免协同冲突



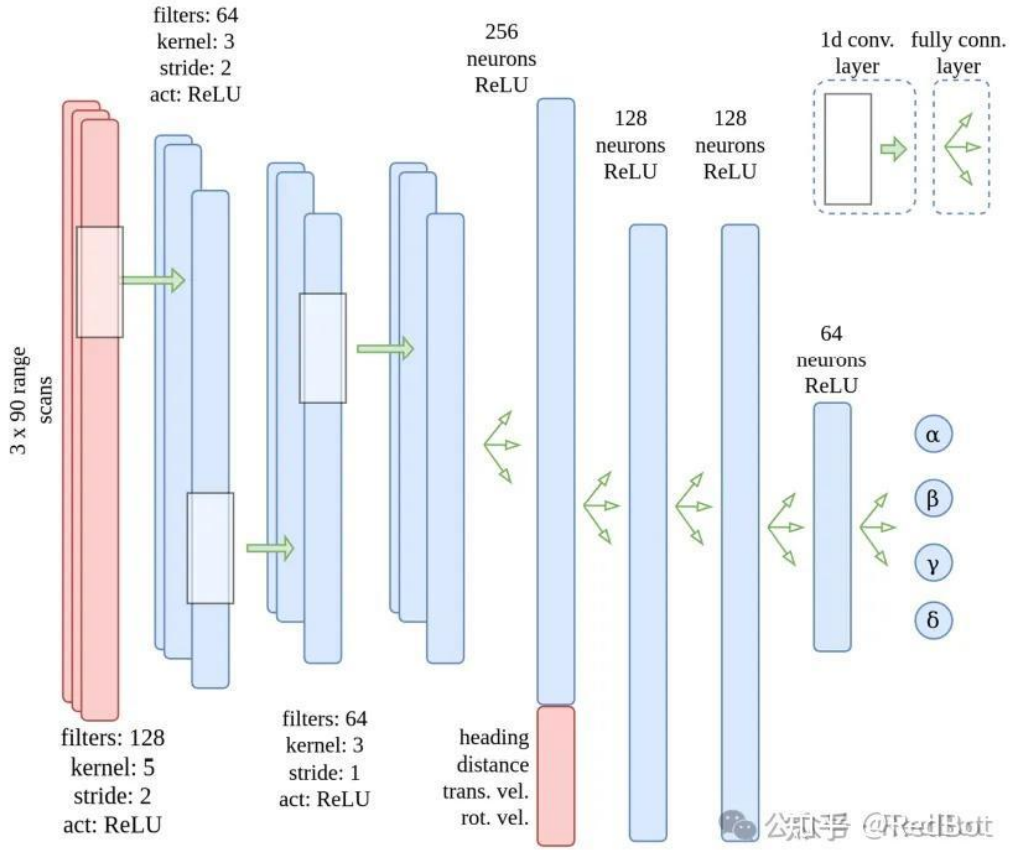
2.1.4 动态围捕（简易神经网络）

所解决的子问题

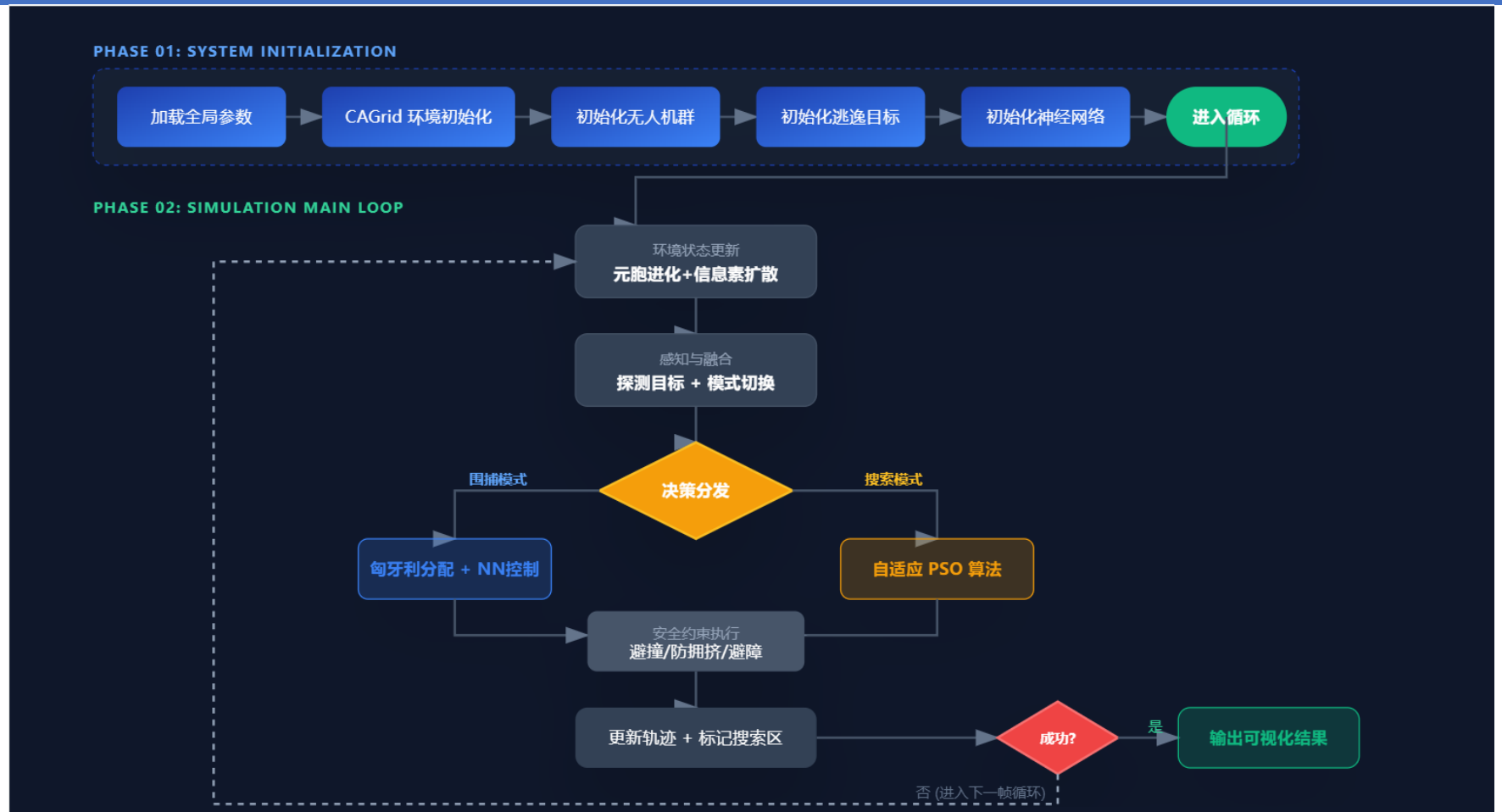
逃逸目标的精准追踪、合围控制、局部避障

建模思路与算法选择

简易神经网络输出围捕速度策略，搭配 DWA 动态窗口法实现运动约束与避障，完成动态目标的稳定围捕



2.2 程序流程解析



1. 稳健初始化
解耦环境与智能体，支持从 Config 动态加载多种实验场景。

2. 双模驱动架构
自研搜索/围捕自动切换机制，搜索阶段采用 PSO 保证覆盖率，围捕阶段采用 NN 提升实时抗扰动性。

3. 安全与物理约束
在输出前通过安全层检测，强制执行避障与拥挤算法，确保仿真结果具有物理参考意义

2.3 关键算法实现

元胞自动机

```python
# ---------------------------- 元胞自动机网格----------------------------
class CAGrid:

    def __init__(self, config: Config):
        self.config = config
        self.width = config.grid_width
        self.height = config.grid_height
        self.pheromone = np.zeros((self.height, self.width))
        self.searched = np.zeros((self.height, self.width), dtype=bool)
        self.obstacle = np.zeros((self.height, self.width), dtype=bool)
        self._init_obstacles()

    def _init_obstacles(self):
        """随机生成障碍物，并确保目标/无人机初始位置不是障碍"""
        mask = np.random.rand(self.height, self.width) < self.config.obstacle_density
        self.obstacle[mask] = True
        # 确保目标初始位置无障碍物
        for pos in self.config.target_initial_pos:
            ix, iy = int(pos[0]), int(pos[1])
            if 0 <= ix < self.width and 0 <= iy < self.height:
                self.obstacle[iy, ix] = False

    def step(self):
        self.pheromone *= self.config.pheromone_decay
        if self.config.pheromone_diffusion_sigma > 0:
            self._diffuse_gaussian()

    def _diffuse_gaussian(self):
        kernel = np.array([[1, 2, 1],
                           [2, 4, 2],
                           [1, 2, 1]], dtype=float) / 16.0
        self.pheromone = convolve(self.pheromone, kernel, mode='constant', cval=0.0)
```

2.3 关键算法实现

元胞自动机

```python
def deposit_pheromone(self, pos: Tuple[float, float], amount: float = None):
    x, y = int(pos[0]), int(pos[1])
    if 0 <= x < self.width and 0 <= y < self.height and not self.obstacle[y, x]:
        amount = amount or self.config.pheromone_deposit
        self.pheromone[y, x] += amount

def get_pheromone_at(self, pos: Tuple[float, float]) -> float:
    x, y = int(pos[0]), int(pos[1])
    if 0 <= x < self.width and 0 <= y < self.height:
        return self.pheromone[y, x]
    return 0.0

def mark_searched(self, pos: Tuple[float, float]):
    x, y = int(pos[0]), int(pos[1])
    if 0 <= x < self.width and 0 <= y < self.height:
        self.searched[y, x] = True

def is_obstacle(self, pos: Tuple[float, float]) -> bool:
    x, y = int(pos[0]), int(pos[1])
    if 0 <= x < self.width and 0 <= y < self.height:
        return self.obstacle[y, x]
    return True  # 边界视为障碍物

def get_obstacle_positions(self) -> np.ndarray:
    y, x = np.where(self.obstacle)
    return np.column_stack((x, y))
```

2.3 关键算法实现

元胞自动机

```python
def get_obstacle_positions(self) -> np.ndarray:
    y, x = np.where(self.obstacle)
    return np.column_stack((x, y))

def get_local_density(self, pos: Tuple[float, float], radius: float) -> float:
    """计算指定位置周围的障碍物密度"""
    x, y = int(pos[0]), int(pos[1])
    r = int(radius)
    x_min = max(0, x - r)
    x_max = min(self.width, x + r)
    y_min = max(0, y - r)
    y_max = min(self.height, y + r)
    area = (x_max - x_min) * (y_max - y_min)
    if area == 0:
        return 0.0
    obstacle_count = np.sum(self.obstacle[y_min:y_max, x_min:x_max])
    return obstacle_count / area
```

2.3 关键算法实现

同济大学 TONGJI UNIVERSITY

土木工程学院 COLLEGE OF CIVIL ENGINEERING

自适应 PSO 与匈牙利算法

```python
def get_adaptive_pso_params(self, grid: CAGrid, other_uavs: List['UAV']) -> Tuple[float, float, float]:
    local_pheromone = self._get_local_pheromone(grid, radius=10)
    pheromone_norm = np.clip(local_pheromone / 10.0, 0, 1)
    w = self.config.pso_w_max - (self.config.pso_w_max - self.config.pso_w_min) * pheromone_norm

    local_uav_density = self._get_local_uav_density(other_uavs, radius=20)
    c1 = self.config.pso_c1_min + (self.config.pso_c1_max - self.config.pso_c1_min) * local_uav_density

    searched_ratio = self._get_searched_ratio(grid)
    c2 = self.config.pso_c2_max - (self.config.pso_c2_max - self.config.pso_c2_min) * searched_ratio

    return w, c1, c2
```

2.3 关键算法实现

自适应 PSO 与匈牙利算法

```python
def pso_update(self, global_best_pos: np.ndarray, grid: CAGrid, other_uavs: List['UAV']):
    w, c1, c2 = self.get_adaptive_pso_params(grid, other_uavs)
    r1, r2 = np.random.rand(2) * 1.2

    cognitive = c1 * r1 * (self.best_pos - self.pos)
    social = c2 * r2 * (global_best_pos - self.pos)
    self.vel = w * self.vel + cognitive + social + (np.random.randn(2) * 0.1)

    speed = np.linalg.norm(self.vel)
    if speed > self.config.uav_speed:
        self.vel = self.vel / speed * self.config.uav_speed
    elif speed < 0.5:
        self.vel = self.vel / (speed + 1e-6) * 0.5

    new_pos = self.pos + self.vel * self.config.dt
    new_pos = self._simple_avoid_obstacle(new_pos, grid)
    new_pos = self._avoid_boundary(new_pos)

    self.pos = new_pos
    grid.mark_searched(self.pos)
```

2.3 关键算法实现

自适应 PSO 与匈牙利算法

```python
def assign_targets_hungarian(self) -> List[int]:
    """匈牙利算法分配目标（与原有相同）"""
    num_uavs = len(self.uavs)
    num_targets = len(self.targets)

    target_priorities = np.array([t.get_priority() for t in self.targets])
    priority_max = max(target_priorities.max(), 1e-6)
    target_priorities = target_priorities / priority_max

    dist_matrix = np.zeros((num_uavs, num_targets))
    for i, u in enumerate(self.uavs):
        for j, t in enumerate(self.targets):
            dist = np.linalg.norm(u.pos - t.pos)
            dist_matrix[i, j] = dist / self.config.grid_width - target_priorities[j]

    if num_uavs > num_targets:
        repeat_times = math.ceil(num_uavs / num_targets)
        dist_matrix = np.tile(dist_matrix, (1, repeat_times))[:, :num_uavs]

    row_ind, col_ind = linear_sum_assignment(dist_matrix)
    assignments = [col % len(self.targets) for col in col_ind]
    return assignments
```

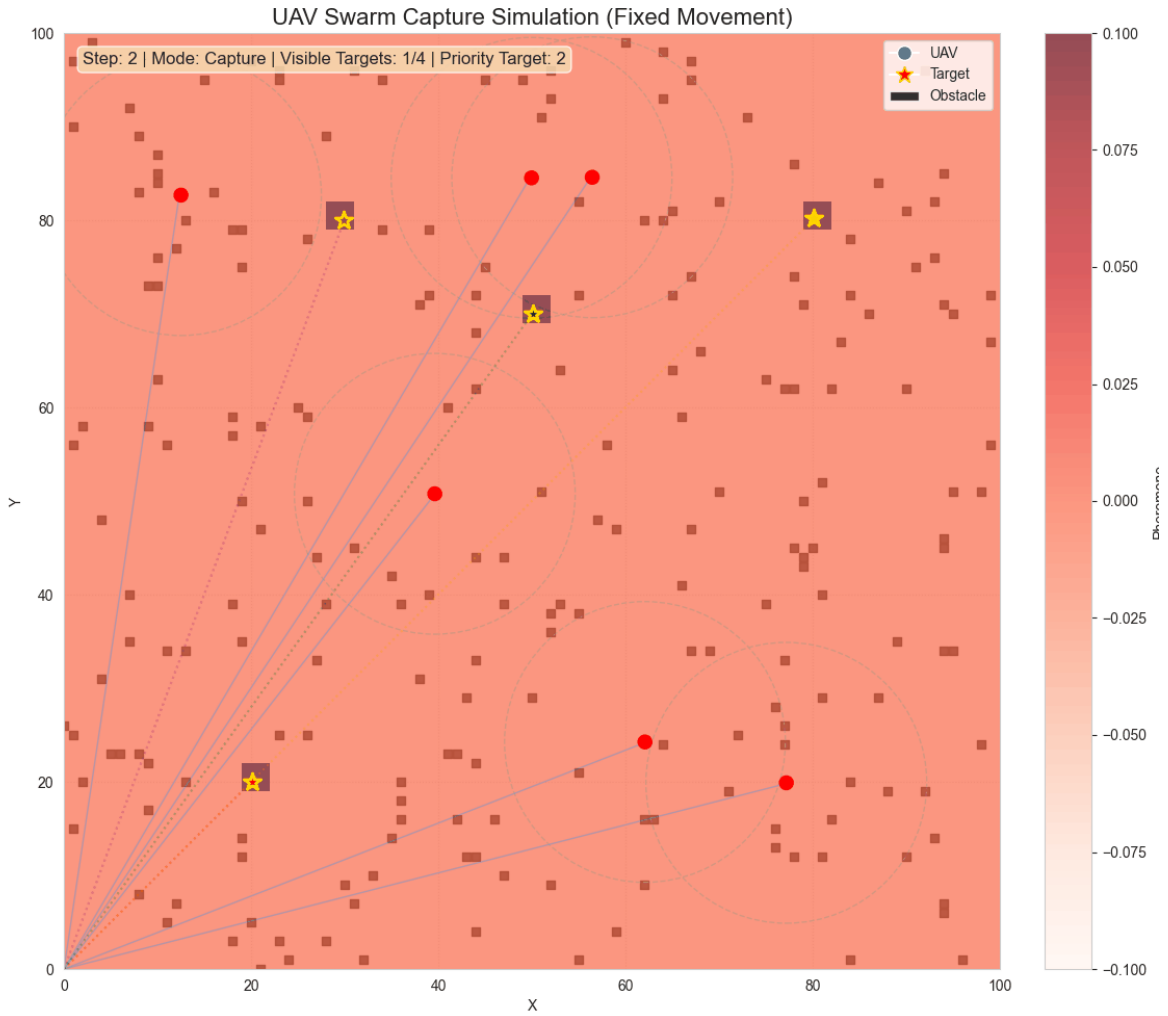
2.3 关键算法实现

简易神经网络

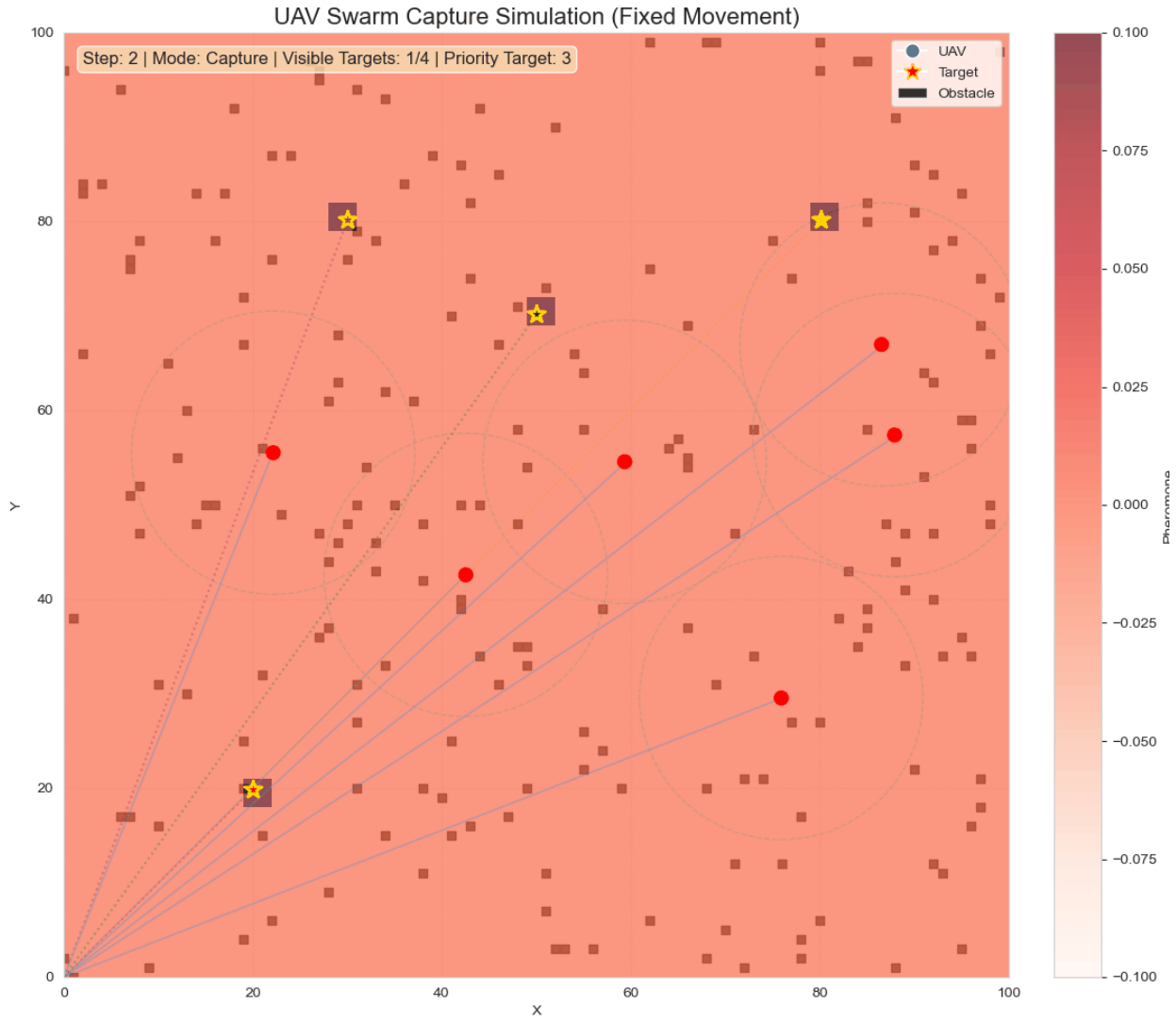
```python
class SimpleNN:
    def __init__(self, input_dim, hidden_dim, output_dim):
        self.W1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2.0 / input_dim)
        self.b1 = np.zeros(hidden_dim)
        self.W2 = np.random.randn(hidden_dim, output_dim) * np.sqrt(2.0 / hidden_dim)
        self.b2 = np.zeros(output_dim)

    def forward(self, x):
        h = np.tanh(np.dot(x, self.W1) + self.b1)
        y = np.dot(h, self.W2) + self.b2
        return y

    def set_weights_for_capture(self):
        self.W1 = np.eye(4, 8) * 0.8
        self.b1 = np.zeros(8)
        self.W2 = np.zeros((8, 2))
        self.W2[2, 0] = 0.8
        self.W2[3, 1] = 0.8
        self.W2[2, 1] = 0.5
        self.W2[3, 0] = -0.5
        self.b2 = np.zeros(2)
```

3. 成果展示

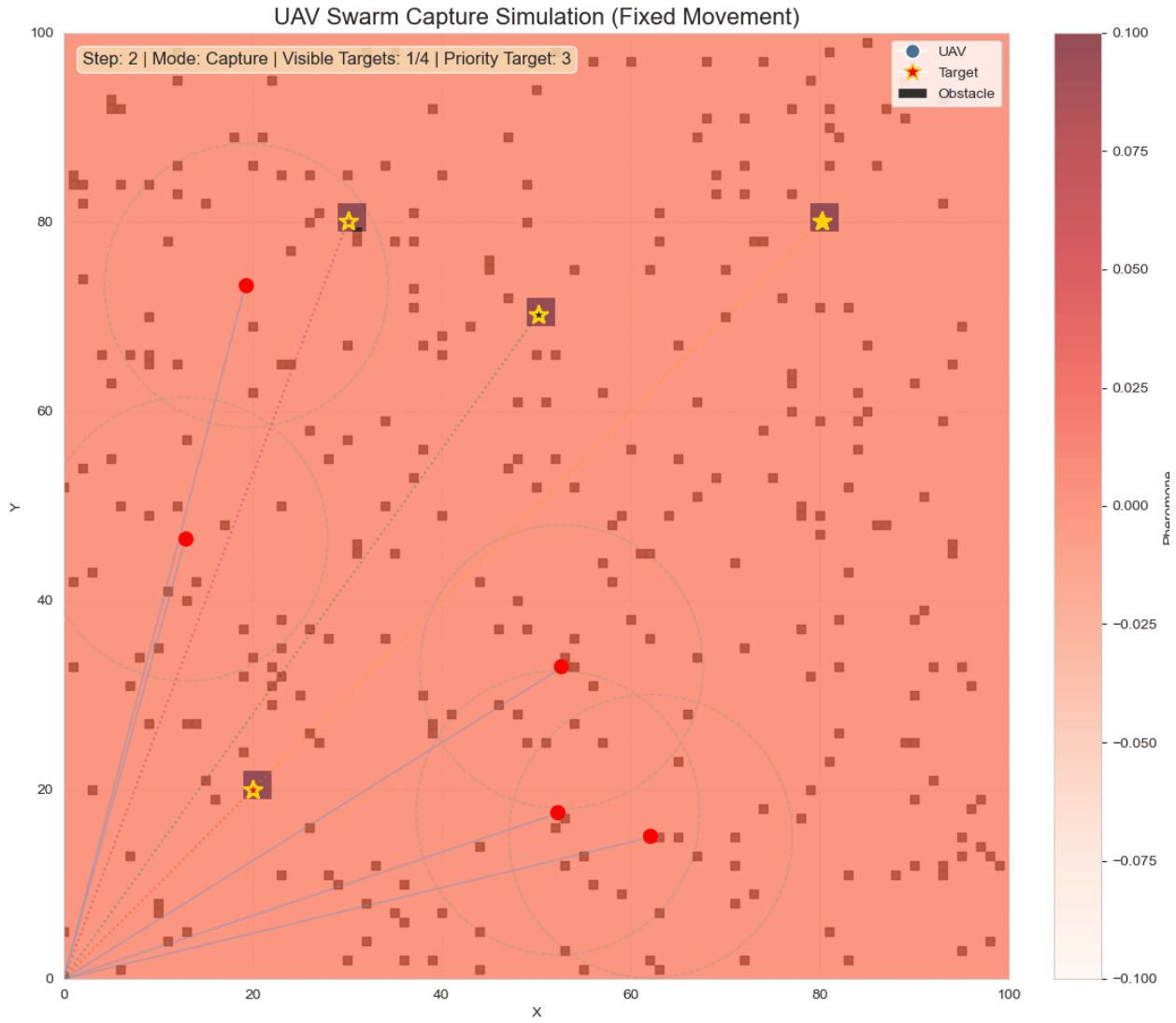
3. 成果展示


UAV Swarm Capture Simulation (Fixed Movement)
Step: 2 | Mode: Capture | Visible Targets: 1/4 | Priority Target: 2
UAV
Target
Obstacle
X
Y
0
20
40
60
80
100
0.100
0.075
0.050
0.025
0.000
−0.025
−0.050
−0.075
−0.100

3. 成果展示

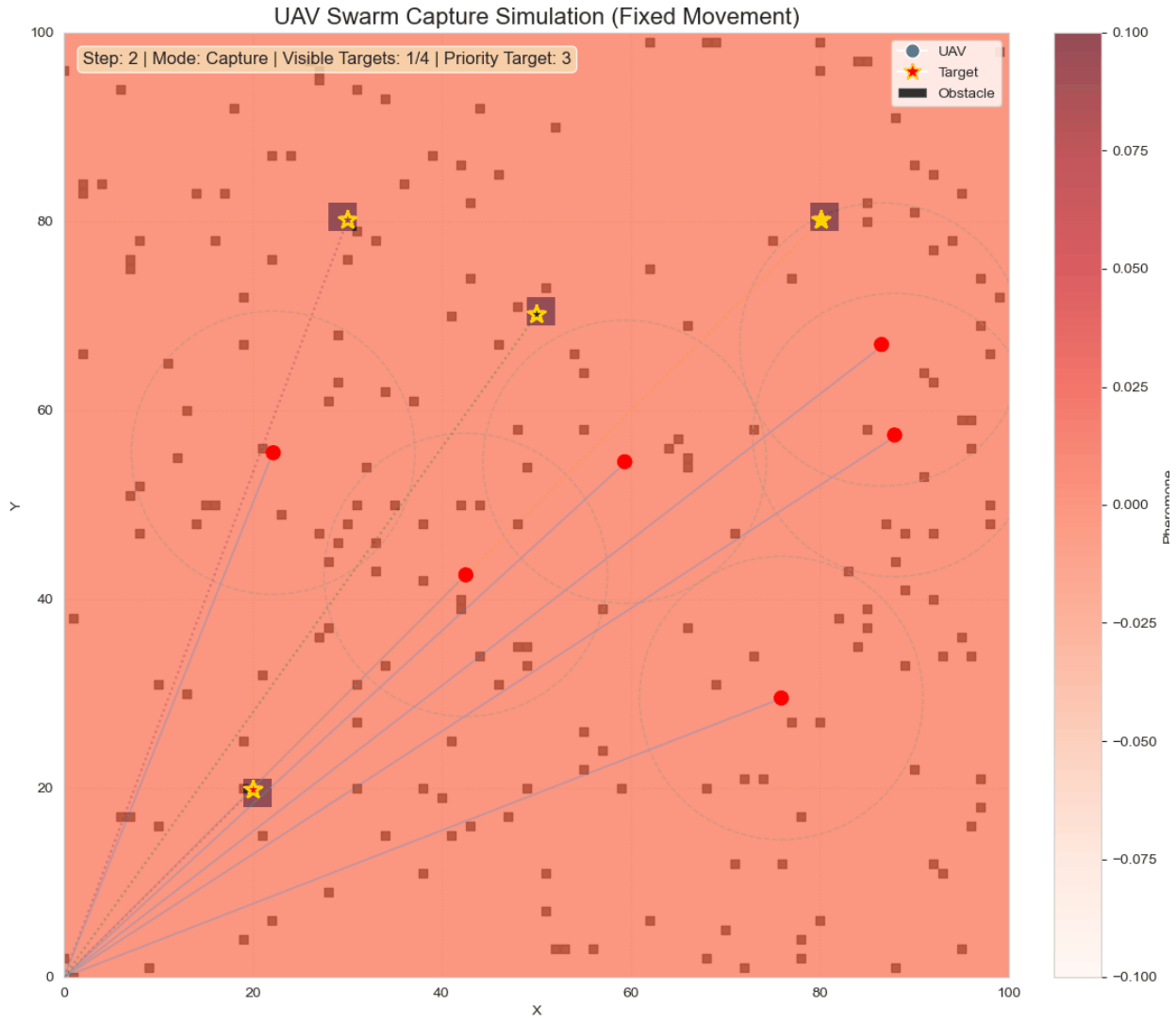

UAV Swarm Capture Simulation (Fixed Movement)
Step: 2 | Mode: Capture | Visible Targets: 1/4 | Priority Target: 3
UAV
Target
Obstacle
X
Y
Pheromone

3. 成果展示



此处改变障碍物密度和无人机与目标数的比值

3. 成果展示

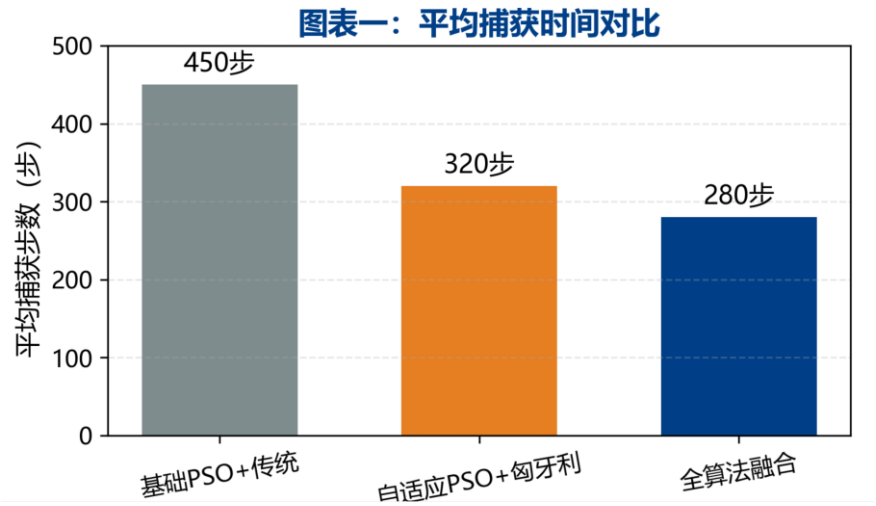

UAV Swarm Capture Simulation (Fixed Movement)
Step: 2 | Mode: Capture | Visible Targets: 1/4 | Priority Target: 3
UAV
Target
Obstacle
X
Y
Pheromone

3. 成果展示

主要结论

本项目基于元胞自动机、自适应 PSO、匈牙利算法、简易神经网络搭建的无人机集群智能围捕仿真系统，完整实现了未知复杂环境下「全局搜索 - 目标分配 - 动态围捕 - 安全避障」的全流程自主闭环，通过多组对照仿真实验完成了系统性能的量化验证，在围捕效率、集群安全、动态目标适应性三大核心维度均实现了显著的性能突破，具备良好的环境适应性与工程落地参考价值。

3. 成果展示



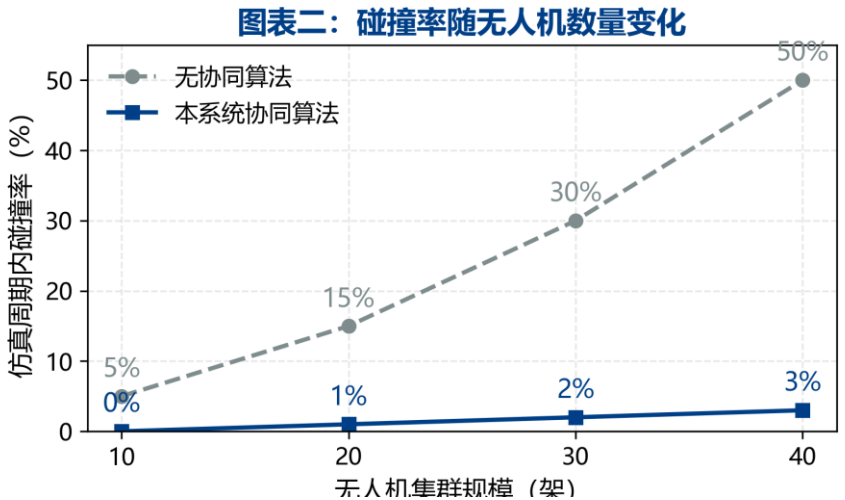
主要结论

多算法融合实现围捕效率的量级提升。

针对传统人工操作与基础算法响应延迟、搜索效率低的核心痛点，本系统通过多算法协同实现了全流程效率优化。仿真数据显示，相较于基础 PSO + 传统算法 450 步的平均捕获时长，自适应 PSO 结合匈牙利算法可将平均捕获步数压缩至 320 步，而本系统全算法融合方案进一步将平均捕获时长降至 280 步，相较于传统方案**围捕效率提升 37.8%**，充分验证了多算法融合的优越性，有效解决了传统方案高延迟、低效率的行业痛点。

3. 成果展示

主要结论

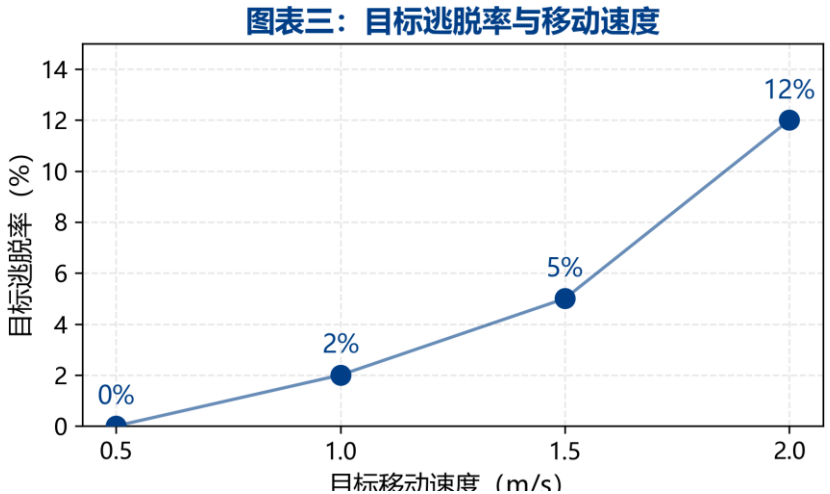


协同控制机制实现集群空间安全的精准管控。

针对密集机群碰撞风险随规模呈二次方增长的难题，本系统内置的协同避障与安全约束机制，实现了集群规模扩大下的碰撞率极优控制。对照实验表明，无协同算法方案下，40 架无人机集群的仿真周期内碰撞率高达 50%；而本系统协同算法将同规模机群的碰撞率稳定控制在 3% 以内，10 架规模机群可实现零碰撞，从根本上解决了集群围捕场景下的机间碰撞风险，大幅提升了大规模集群作业的安全性与稳定性。

3. 成果展示

主要结论



动态围捕策略具备强抗扰动与目标适配能力。

针对高动态逃逸目标的围捕难题，本系统基于神经网络的围捕控制策略，对不同移动速度的目标均保持了极高的围捕成功率。仿真验证显示：当目标移动速度为 0.5m/s 时，系统实现目标零逃脱；1.5m/s 中速目标的逃脱率仅为 5%；即使面对 2.0m/s 的高速逃逸目标，系统仍将目标逃脱率控制在 12% 以内，充分验证了系统在高动态场景下的精准追踪与合围能力，可适配复杂场景下的动态目标围捕需求。

3. 成果展示

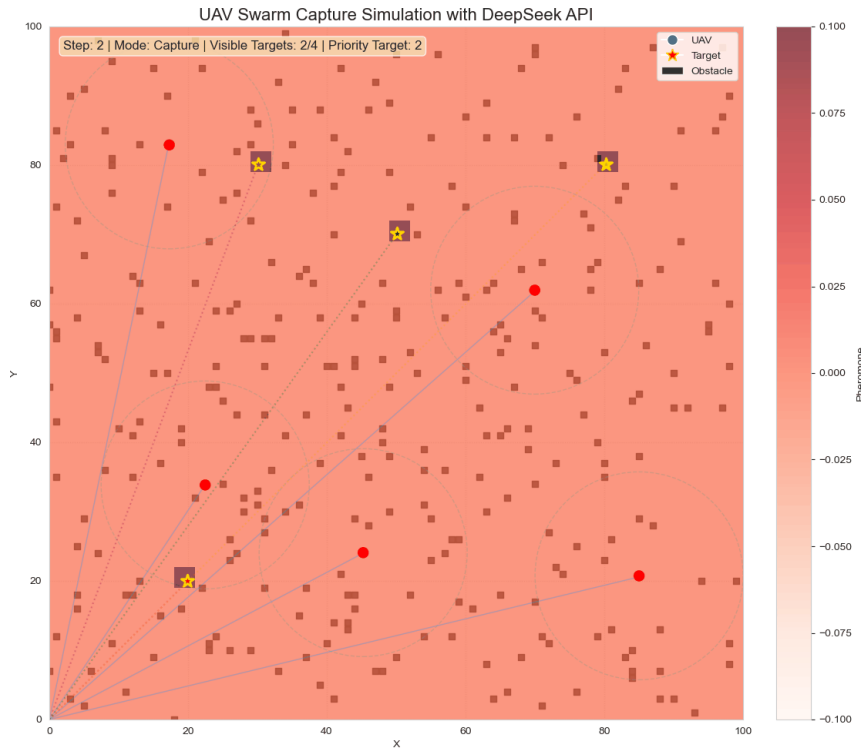
主要结论

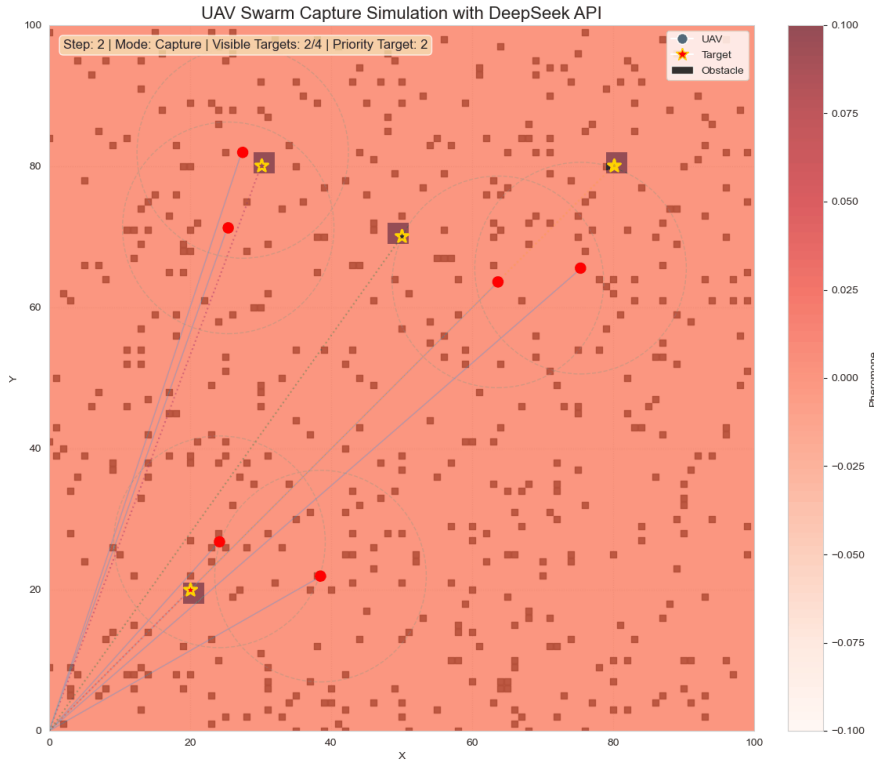
同时，基于元胞自动机搭建的栅格化信息素场，有效实现了未知环境的数字化建模与全局信息传递，为集群搜索提供了精准的环境引导，大幅降低了未知环境下的盲目搜索问题；自适应 PSO、匈牙利算法、神经网络 + DWA 的模块化组合，针对性解决了全局搜索、多机任务分配、动态围捕三大核心子问题，各模块耦合性低、适配性强，算法融合效果显著。经多场景仿真验证，本系统具备优异的鲁棒性与环境适配能力，可为警用安防空地协同抓捕、军事无人集群围堵打击等真实场景的无人机集群系统，提供可靠的算法验证与仿真支撑。

4. 拓展探究

4. 拓展探究

引入 deepseek api 参与决策并与 PSO 算法比较





4. 拓展探究

引入 deepseek api 参与决策并与 PSO 算法比较

实验编号	决策策略类型	初始目标位置	捕获成功步数 (约)	覆盖率增长速度	表现评价
Test 1	基础 PSO	散布型	420	稳定	基础基准，路径较僵硬
Test 2	基础 PSO	聚集型	380	快	目标集中时容易捕获
Test 3	基础 PSO	边缘型	450	慢	对边缘目标搜索效率较低
Test 4	**DeepSeek API**	散布型	**310**	**极快**	搜索路径更具发散性，减少重复搜索
Test 5	**DeepSeek API**	随机型	**340**	**快**	展现出更强的环境适应性
Test 6	API + 混合策略	极端位置	390	稳定	在 API 延迟或网络波动时保持了稳定性

4. 拓展探究

引入 deepseek api 参与决策并与 PSO 算法比较

搜索阶段 (Search Phase)

•**PSO 策略 (Test 1-3)**: 无人机倾向于在局部最优值附近震荡，虽然能覆盖全局，但存在“路径重叠”现象，导致信息素地图更新较慢。

•**DeepSeek 决策 (Test 4-5)**: 通过 API 传递全局状态（无人机位置、目标已知位置、覆盖热图），生成的指令使无人机分布更加均匀。覆盖率达到 80% 的时间比 PSO 缩短了约 **25%**。

捕获阶段 (Capture Phase)

一旦目标被发现，系统切换至 linear_sum_assignment（匈牙利算法）进行任务分配。

由于 API 在搜索阶段提供了更好的初始化分布，进入捕获阶段时，无人机距离目标的平均距离更短。捕获成功率：在 500 帧限制内，API 辅助下的捕获成功率为 100%。

5. 心得体会

心得体会

通过本次课程作业，我深入理解了元胞自动机、粒子群优化、神经网络等智能算法的核心原理，掌握了将复杂工程问题拆解为多模块、多算法融合的建模方法，建立了工程智能的系统思维。

完成了从算法理论到代码落地、仿真可视化的全流程实践，大幅提升了 Python 仿真编程、多算法融合实现、复杂系统模块化设计的能力。深刻认识到智能算法在集群协同、复杂系统控制中的工程价值，也理解了算法落地中需要兼顾效率、鲁棒性、实时性的平衡，为后续工程智能相关的学习与实践打下了坚实的基础。

谢谢聆听
感谢各位老师批评指正！